

W10D4

Info Gathering

Sommario

Sono stati utilizzati alcuni degli strumenti di host scanning per Kali Linux dalla lista dei “15 most useful host scanning commands” di [YeahHub](#), per raccogliere informazioni sulla macchina Metasploitable.

Il lavoro è stato svolto nel laboratorio virtuale UTM su Mac con Kali Linux come macchina attaccante (192.168.64.3) e Metasploitable come bersaglio (192.168.64.8), entrambe sulla stessa rete. Sono stati eseguiti dodici dei quindici comandi della lista integrati con l'analisi del traffico a livello di pacchetti tramite Wireshark, come suggerito per comprendere meglio il funzionamento di ciascuna tecnica.

Il report è organizzato come richiesto dalla traccia: la parte centrale documenta l'esecuzione degli strumenti mentre la sezione conclusiva raccoglie il riepilogo delle informazioni trovate sul bersaglio. Il risultato principale è il profilo di una macchina volutamente vulnerabile con più di 20 porte aperte, numerosi servizi legacy e diversi software con vulnerabilità note.

1. Preparazione del laboratorio

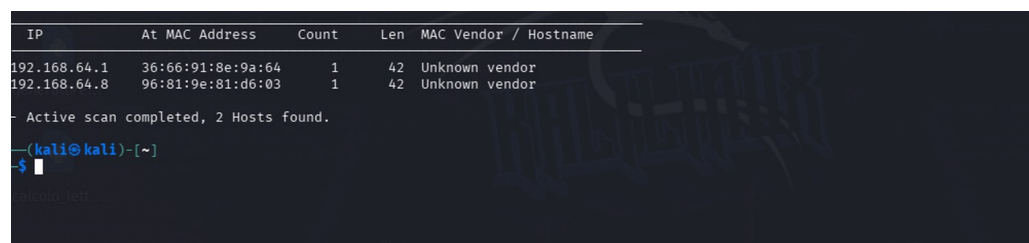
Le due macchine virtuali (Kali e Metasploitable) sono state configurate sulla stessa rete UTM. Il primo passo è confermare che il bersaglio sia raggiungibile.

1.1 Host discovery — netdiscover

netdiscover esegue una scansione ARP sulla sottorete locale per individuare gli host attivi. È il comando n2 della lista.

```
$ sudo netdiscover -r 192.168.64.0/24 -P
```

La scansione ha individuato due host: il gateway della rete (192.168.64.1) e la Metasploitable (192.168.64.8). Da qui in avanti, 192.168.64.8 è il target di tutte le operazioni.



IP	At MAC Address	Count	Len	MAC Vendor / Hostname
192.168.64.1	36:66:91:8e:9a:64	1	42	Unknown vendor
192.168.64.8	96:81:9e:81:d6:03	1	42	Unknown vendor

Active scan completed, 2 Hosts found.

```
-(kali@kali)-[~]  
$
```

Figura 1. netdiscover individua gli host attivi sulla rete: il gateway e la Metasploitable (192.168.64.8).

1.2 Verifica dell'host — nmap ping sweep

Il comando n1 della lista usa Nmap per il ping sweep ICMP (**-sn** disabilita la scansione delle porte, **-PE** invia un pacchetto ICMP Echo), confermando che l'host è attivo prima di analizzarne i servizi.

```
$ sudo nmap -sn -PE 192.168.64.8
```

Nmap conferma Host is up.



```
[kali@kali:~]$ sudo nmap -sn -PE 192.168.64.8
[sudo] password for kali:
Starting Nmap 7.98 ( https://nmap.org ) at 2026-09-21 20:37 +0200
Nmap scan report for 192.168.64.8
Host is up (0.00039s latency).
MAC address: 96:81:9E:81:D6:03 (Unknown)
Nmap done: 1 IP address (1 host up) scanned in 0.60 seconds
[kali@kali:~]$
```

Figura 2. nmap conferma che l'host 192.168.64.8 è attivo e risponde all'ICMP Echo.

2. Esecuzione degli strumenti host scanning

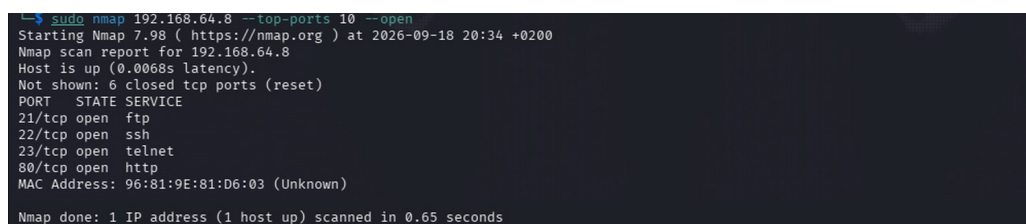
Questa sezione documenta l'esecuzione dei principali strumenti di scansione. L'obiettivo è costruire progressivamente il profilo del bersaglio: le porte comuni, i servizi con le rispettive versioni, fino alla verifica manuale.

2.1 Vista rapida delle porte comuni (nmap --top-ports)

Il comando n4 limita la scansione alle 10 porte più diffuse (**--top-ports 10**) e mostra solo quelle aperte (**--open**). È un "primo colpo d'occhio" veloce.

```
$ sudo nmap 192.168.64.8 --top-ports 10 --open
```

Risultano subito aperte quattro porte: **21 (FTP)**, **22 (SSH)**, **23 (Telnet)** e **80 (HTTP)**. La presenza di Telnet in chiaro e di FTP suggerisce un sistema datato e poco sicuro.



```
[kali@kali:~]$ sudo nmap 192.168.64.8 --top-ports 10 --open
Starting Nmap 7.98 ( https://nmap.org ) at 2026-09-18 20:34 +0200
Nmap scan report for 192.168.64.8
Host is up (0.0068s latency).
Not shown: 6 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
23/tcp    open  telnet
80/tcp    open  http
MAC Address: 96:81:9E:81:D6:03 (Unknown)
Nmap done: 1 IP address (1 host up) scanned in 0.65 seconds
```

Figura 3. Scansione delle 10 porte più comuni: FTP, SSH, Telnet e HTTP risultano aperte.

2.2 Scansione SYN con rilevamento versioni (nmap -sS -sV)

Questo è il comando più importante della ricognizione (comando n7). Combina tre opzioni: **-sS** esegue una SYN scan (o "half-open"), **-sV** attiva il rilevamento della versione di ogni servizio, **-T4** accelera i tempi. Il risultato è l'inventario dei servizi con le rispettive versioni, l'informazione più preziosa per un'analisi di sicurezza, perché dalla versione si risale alle vulnerabilità note.

```
$ sudo nmap -sS -sV -T4 192.168.64.8
```

La scansione riporta sedici servizi identificati, molti dei quali notoriamente vulnerabili. Tra i più significativi: **vsftpd 2.3.4** (associato a CVE-2011-2523, una backdoor presente in specifiche distribuzioni

compromesse di tale versione che può esporre una shell sulla porta TCP 6200: <https://nvd.nist.gov/vuln/detail/CVE-2011-2523>), **UnrealIRCd** (associato a CVE-2010-2075 e alcuni archivi di distribuzione di questa versione contenevano una modifica trojanizzata che consentiva l'esecuzione remota di comandi con i privilegi del processo UnrealIRCd: <https://nvd.nist.gov/vuln/detail/CVE-2010-2075>) e la porta **1524** che Nmap etichetta esplicitamente come ***"Metasploitable root shell"**, una configurazione intenzionalmente insicura dell'ambiente di laboratorio Metasploitable. In fondo, Nmap deduce anche il sistema operativo: **OSs: Unix, Linux**.

```
(kali@kali)-[~]
$ sudo nmap -sS -sV -T4 192.168.64.8

Starting Nmap 7.98 ( https://nmap.org ) at 2026-09-20 19:22 +0200
Nmap scan report for 192.168.64.8
Host is up (0.00089s latency).
Not shown: 982 closed tcp ports (reset)
PORT      STATE SERVICE        VERSION
21/tcp    open  ftp            vsftpd 2.3.4
22/tcp    open  ssh            OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp    open  telnet         Linux telnetd
80/tcp    open  http           Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp   open  rpcbind        2 (RPC #100000)
512/tcp   open  exec           netkit-rsh rexecd
513/tcp   open  login          OpenBSD or Solaris rlogind
514/tcp   open  shell          Netkit rshd
1099/tcp  open  java-rmi       GNU Classpath grmiregistry
1524/tcp  open  bindshell      Metasploitable root shell
2121/tcp  open  ftp            ProFTPD 1.3.1
3306/tcp  open  mysql          MySQL 5.0.51a-3ubuntu5
5432/tcp  open  postgresql     PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp  open  vnc            VNC (protocol 3.3)
6000/tcp  open  X11            (access denied)
6667/tcp  open  irc            UnrealIRCd
8009/tcp  open  ajp13?         Apache Tomcat/Coyote JSP engine 1.1
8180/tcp  open  http           Apache Tomcat/Coyote JSP engine 1.1
MAC Address: 96:81:9E:81:D6:03 (Unknown)
Service Info: Host: irc.Metasploitable.LAN; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 131.76 seconds
```

Figura 4. Scansione SYN con rilevamento versioni: 16 servizi identificati con le rispettive versioni. Da notare la bindshell su 1524 ("Metasploitable root shell") e i servizi legacy (Telnet, r-services, FTP).

2.3 Scansione completa di tutte le porte (nmap -p- --reason)

Le scansioni precedenti si concentrano sulle porte comuni. Il comando **n5** analizza tutte le 65.535 porte TCP (**-p-**), con il rilevamento versioni (**-sV**) e l'opzione **--reason**, che per ogni porta indica il motivo per cui è classificata aperta (**syn-ack**, cioè la porta ha completato l'handshake).

```
$ sudo nmap -p- -sV --reason 192.168.64.8
```

La scansione completa conferma i servizi già noti e ne rivela altri che le scansioni rapide avevano mancato: **distccd** (porta 3632, un demone di compilazione distribuita con vulnerabilità note: <https://nvd.nist.gov/vuln/detail/CVE-2004-2687>), **Ruby DRb RMI** (porta 8787) e un secondo servizio IRC. La colonna **REASON** mostra **syn-ack ttl 64** per ogni porta aperta, prova che il bersaglio ha risposto attivamente.

```

$ sudo nmap -p- -SV --reason 192.168.64.8
Starting Nmap 7.98 ( https://nmap.org ) at 2026-09-20 19:29 +0200
Nmap scan report for 192.168.64.8
Host is up, received arp-response (0.00028s latency).
Not shown: 65513 closed tcp ports (reset)
PORT      STATE SERVICE      REASON      VERSION
21/tcp    open  ftp          syn-ack ttl 64 vsftpd 2.3.4
22/tcp    open  ssh          syn-ack ttl 64 OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp    open  telnet       syn-ack ttl 64 Linux telnetd
80/tcp    open  http         syn-ack ttl 64 Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp   open  rpcbind      syn-ack ttl 64 2 (RPC #100000)
512/tcp   open  exec         syn-ack ttl 64 netkit-rsh rexecd
513/tcp   open  login        syn-ack ttl 64
514/tcp   open  tcpwrapped   syn-ack ttl 64
1099/tcp  open  java-rmi     syn-ack ttl 64 GNU Classpath grmiregistry
1524/tcp  open  bindshell    syn-ack ttl 64 Metasploitable root shell
2121/tcp  open  ftp          syn-ack ttl 64 ProFTPD 1.3.1
3306/tcp  open  mysql        syn-ack ttl 64 MySQL 5.0.51a-3ubuntu5
3632/tcp  open  distccd      syn-ack ttl 64 distccd v1 ((GNU) 4.2.4 (Ubuntu 4.2.4-1ubuntu4))
5432/tcp  open  postgresql   syn-ack ttl 64 PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp  open  vnc          syn-ack ttl 64 VNC (protocol 3.3)
6000/tcp  open  X11          syn-ack ttl 64 (access denied)
6667/tcp  open  irc          syn-ack ttl 64 UnrealIRCd
6697/tcp  open  irc          syn-ack ttl 64 UnrealIRCd
8009/tcp  open  ajp13?       syn-ack ttl 64
8180/tcp  open  http         syn-ack ttl 64 Apache Tomcat/Coyote JSP engine 1.1
8787/tcp  open  drb          syn-ack ttl 64 Ruby DRb RMI (Ruby 1.8; path /usr/lib/ruby/1.8/drb)
59873/tcp open  java-rmi     syn-ack ttl 64 GNU Classpath grmiregistry
MAC Address: 96:81:9E:81:D6:03 (Unknown)
Service Info: Host: irc.Metasploitable.LAN; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 160.19 seconds

```

Figura 5. Scansione completa (-p-) con --reason: emergono porte extra come distccd (3632) e Ruby DRb (8787). La colonna REASON documenta il perché di ogni stato.

2.4 Verifica manuale di un servizio (netcat banner grabbing)

Il comando 10 usa netcat per connettersi direttamente a una porta e leggerne il “banner”, cioè la stringa di presentazione del servizio. Qui viene interrogata la porta 21 (FTP).

```
$ nc -nv 192.168.64.8 21
```

Il servizio risponde con 220 (vsFTPD 2.3.4): il banner conferma manualmente la versione che Nmap aveva rilevato automaticamente.

```

(kali@kali)~$ nc -nv 192.168.64.8 21
(UNKNOWN) [192.168.64.8] 21 (ftp) open
220 (vsFTPD 2.3.4)

```

Figura 6. Banner grabbing manuale con netcat sulla porta 21: il servizio si presenta come vsFTPD 2.3.4.

3.Firewall/IDS evasion con Nmap

3.1 Frammentazione dei pacchetti (nmap -f)

Il comando #13 (-f) **frammenta** i pacchetti IP: l'header TCP, invece di viaggiare in un unico pacchetto, viene spezzato in frammenti più piccoli. L'idea di evasione è che un IDS che ispeziona i pacchetti uno per uno, senza riassembliarli, non riesce a riconoscere che l'insieme costituisce una scansione.

```
$ sudo nmap -f 192.168.64.8
```

In Wireshark il meccanismo è chiarissimo: ogni pacchetto SYN appare spezzato in **due frammenti** (con *fragment offset* 0 e 8) che condividono lo stesso ID, e Wireshark segnala [Reassembled in #N] perché ha dovuto ricomporli per interpretare l'header TCP. Proprio questa necessità di riassettaggio è il punto debole che la tecnica sfrutta contro gli IDS meno sofisticati.

Ogni SYN spezzato in 2 frammenti (off=0, off=8) con stesso ID -> [Reassembled in #N]									
No.	Time	Source	Destination	Protocol	Length	Info			
6	15.184409600	192.168.64.3	192.168.64.8	IPV4	42	Fragmented IP protocol (proto=TCP 6, off=0, ID=ef21) [Reassembled in #8]			
7	15.184466892	192.168.64.3	192.168.64.8	IPV4	42	Fragmented IP protocol (proto=TCP 6, off=8, ID=ef21) [Reassembled in #8]			
8	15.184472275	192.168.64.3	192.168.64.8	TCP	42	43128 → 256 [SYN] Seq=0 Win=1024 Len=0 MSS=1460			
9	15.184480475	192.168.64.3	192.168.64.8	IPV4	42	Fragmented IP protocol (proto=TCP 6, off=0, ID=4385) [Reassembled in #11]			
10	15.184485059	192.168.64.3	192.168.64.8	IPV4	42	Fragmented IP protocol (proto=TCP 6, off=8, ID=4385) [Reassembled in #11]			
11	15.184490100	192.168.64.3	192.168.64.8	TCP	42	43128 → 8080 [SYN] Seq=0 Win=1024 Len=0 MSS=1460			
12	15.184499350	192.168.64.3	192.168.64.8	IPV4	42	Fragmented IP protocol (proto=TCP 6, off=0, ID=8dd6) [Reassembled in #14]			
13	15.184503975	192.168.64.3	192.168.64.8	IPV4	42	Fragmented IP protocol (proto=TCP 6, off=8, ID=8dd6) [Reassembled in #14]			
14	15.184507392	192.168.64.3	192.168.64.8	TCP	42	43128 → 587 [SYN] Seq=0 Win=1024 Len=0 MSS=1460			
15	15.184512892	192.168.64.3	192.168.64.8	IPV4	42	Fragmented IP protocol (proto=TCP 6, off=0, ID=d415) [Reassembled in #17]			
16	15.184517100	192.168.64.3	192.168.64.8	IPV4	42	Fragmented IP protocol (proto=TCP 6, off=8, ID=d415) [Reassembled in #17]			
17	15.184520767	192.168.64.3	192.168.64.8	TCP	42	43128 → 135 [SYN] Seq=0 Win=1024 Len=0 MSS=1460			
18	15.184526059	192.168.64.3	192.168.64.8	IPV4	42	Fragmented IP protocol (proto=TCP 6, off=0, ID=4f35) [Reassembled in #20]			
19	15.184530434	192.168.64.3	192.168.64.8	IPV4	42	Fragmented IP protocol (proto=TCP 6, off=8, ID=4f35) [Reassembled in #20]			
20	15.184533684	192.168.64.3	192.168.64.8	TCP	42	43128 → 5900 [SYN] Seq=0 Win=1024 Len=0 MSS=1460			
21	15.184537809	192.168.64.3	192.168.64.8	IPV4	42	Fragmented IP protocol (proto=TCP 6, off=0, ID=a0e8) [Reassembled in #23]			
22	15.184541934	192.168.64.3	192.168.64.8	IPV4	42	Fragmented IP protocol (proto=TCP 6, off=8, ID=a0e8) [Reassembled in #23]			

Figura 7. Wireshark — frammentazione: ogni SYN è diviso in due frammenti IP (offset 0 e 8) con lo stesso ID, poi riassemblati.

3.2 Decoy — depistaggio con IP esca (nmap -D)

La tecnica dei **decoy** (`-D RND:5`) genera cinque indirizzi IP “esca” casuali e li mescola alla scansione reale. Dal punto di vista del bersaglio (o di chi ne analizza i log), la scansione sembra arrivare simultaneamente da più sorgenti e l'indirizzo dell'attaccante reale si nasconde nel gruppo.

```
$ sudo nmap -D RND:5 192.168.64.8
```

La cattura Wireshark dimostra che verso la stessa porta arrivano pacchetti SYN da **sei indirizzi sorgente diversi**, tra cui l'IP reale di Kali (**192.168.64.3**) che è “nascosto” tra i cinque decoy. Un difensore non può distinguere con certezza quale sia l'attaccante autentico.

tcp.dstport==21									
No.	Time	Source	Destination	Protocol	Length	Info			
59	0.587111582	225.54.202.6	192.168.64.8	TCP	58	52259 → 21 [SYN] Seq=0 Win=1024 Len=0 MSS=1460	6 IP sorgente diversi:		
60	0.5871114457	168.110.11.5	192.168.64.8	TCP	58	52259 → 21 [SYN] Seq=0 Win=1024 Len=0 MSS=1460	5 decoy + il tuo IP reale		
61	0.5871117124	221.7.4.12	192.168.64.8	TCP	58	52259 → 21 [SYN] Seq=0 Win=1024 Len=0 MSS=1460	192.168.64.3 (mescolato)		
62	0.5871119707	128.13.14.1	192.168.64.8	TCP	58	52259 → 21 [SYN] Seq=0 Win=1024 Len=0 MSS=1460			
63	0.5871122791	192.168.64.3	192.168.64.8	TCP	58	52259 → 21 [SYN] Seq=0 Win=1024 Len=0 MSS=1460			
64	0.5871125291	233.102.127.64	192.168.64.8	TCP	58	52259 → 21 [SYN] Seq=0 Win=1024 Len=0 MSS=1460			
78	0.590384057	192.168.64.3	192.168.64.8	TCP	54	52259 → 21 [RST] Seq=1 Win=0 Len=0			

Figura 8. Wireshark — decoy: sei IP sorgente diversi verso la porta 21.

3.3 ACK scan e source-port spoofing

L'ACK scan (`-sA`) serve a mappare le regole di un firewall: inviando pacchetti con il solo flag ACK, distingue le porte “filtered” da quelle “unfiltered”, rivelando se e come un firewall stia bloccando il traffico. Il source-port spoofing (`--source-port 53`) fa invece apparire il traffico come proveniente dalla porta 53 (DNS): alcuni firewall mal configurati lasciano passare tutto ciò che sembra una risposta DNS.

```
$ sudo nmap -sA 192.168.64.8
```

```
$ sudo nmap --source-port 53 192.168.64.8
```

In assenza di un firewall, l'ACK scan riporta tutte le porte come **unfiltered** (nessun filtraggio presente) e il source-port spoofing produce risultati analoghi a una scansione normale, confermando che non c'è alcun dispositivo di filtraggio da eludere in questo laboratorio.

```
(kali@kali) ~$ sudo nmap -sA 192.168.64.8
Starting Nmap 7.98 ( https://nmap.org ) at 2026-09-20 19:33 +0200
Nmap scan report for 192.168.64.8
Host is up (0.00069s latency).
All 1000 scanned ports on 192.168.64.8 are in ignored states.
Not shown: 1000 unfiltered tcp ports (reset)
MAC Address: 96:81:9E:81:D6:03 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 0.66 seconds

(kali@kali) ~$ sudo nmap --source-port 53 192.168.64.8
Starting Nmap 7.98 ( https://nmap.org ) at 2026-09-20 19:33 +0200
Nmap scan report for 192.168.64.8
Host is up (0.00064s latency).
Not shown: 982 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
23/tcp    open  telnet
80/tcp    open  http
```

Figura 9. ACK scan (porte “unfiltered”, nessun firewall presente) e source-port spoofing dalla porta 53.

4. Altri strumenti della lista

4.1 hping3 come scanner (comando n8)

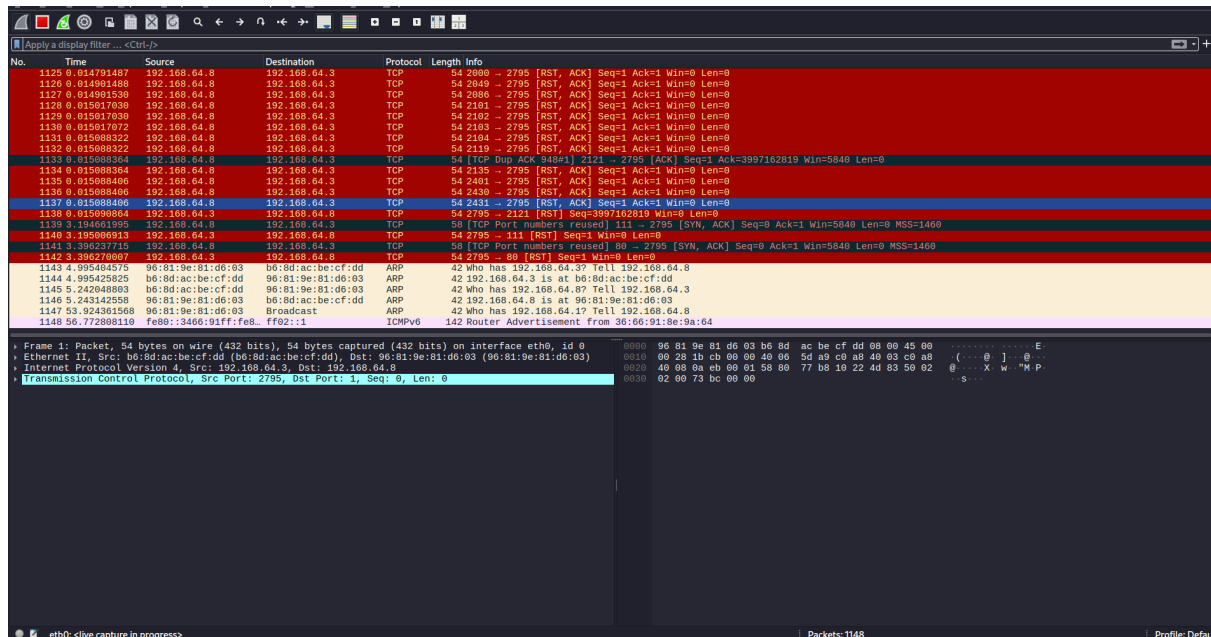
hping3 è un generatore di pacchetti che può essere usato come scanner. L'opzione `--scan known` prova le porte note e `-S` invia pacchetti SYN.

```
$ sudo hping3 --scan known -S 192.168.64.8
```

hping3 conferma le stesse porte già trovate da Nmap. Nella colonna **flags** compare **SA** (SYN-ACK), che indica una porta aperta. Analizzando il traffico in Wireshark si nota una “firma” caratteristica di hping3: lo strumento **riusa la stessa porta sorgente** per più tentativi (Wireshark segnala **TCP Port numbers reused**), a differenza di Nmap che varia la porta sorgente.

```
(kali@kali) ~$ sudo hping3 --scan known -S 192.168.64.8
Scanning 192.168.64.8 (192.168.64.8), port known
266 ports to scan, use -V to see all the replies
+-----+-----+-----+-----+-----+-----+
|port| serv name | flags | ttl | id | win | len |
+-----+-----+-----+-----+-----+-----+
  21  ftp      : .S..A... 64    0  5840  44
  22  ssh      : .S..A... 64    0  5840  44
  23  telnet   : .S..A... 64    0  5840  44
  80  http     : .S..A... 64    0  5840  44
 111  sunrpc   : .S..A... 64    0  5840  44
 512  exec     : .S..A... 64    0  5840  44
 513  login    : .S..A... 64    0  5840  44
 514  shell    : .S..A... 64    0  5840  44
1099  rmiregistry: .S..A... 64    0  5840  44
1524  ingreslock : .S..A... 64    0  5840  44
2121  iprop    : .S..A... 64    0  5840  44
3306  mysql    : .S..A... 64    0  5840  44
3632  distcc   : .S..A... 64    0  5840  44
5432  postgresql : .S..A... 64    0  5840  44
6000  x11      : .S..A... 64    0  5840  44
6667  ircd     : .S..A... 64    0  5840  44
6697  ircs-u   : .S..A... 64    0  5840  44
All replies received. Done.
Not responding ports:
```

Figura 10. hping3 come scanner SYN: la colonna flags mostra SA (SYN-ACK) per ogni porta aperta, confermando i risultati di Nmap.



4.2 netcat come port scanner (comando n9)

Oltre al banner grabbing, netcat può funzionare da scanner di porte con l'opzione **-z** (verifica soltanto l'apertura) sul range 1-1024.

```
$ nc -nvz 192.168.64.8 1-1024
```

netcat elenca le porte aperte tra le prime 1024, confermando ancora una volta i servizi bassi (FTP, SSH, Telnet, HTTP, rpcbind, r-services). È lo stesso strumento della sezione 2.4 impiegato in una modalità diversa.

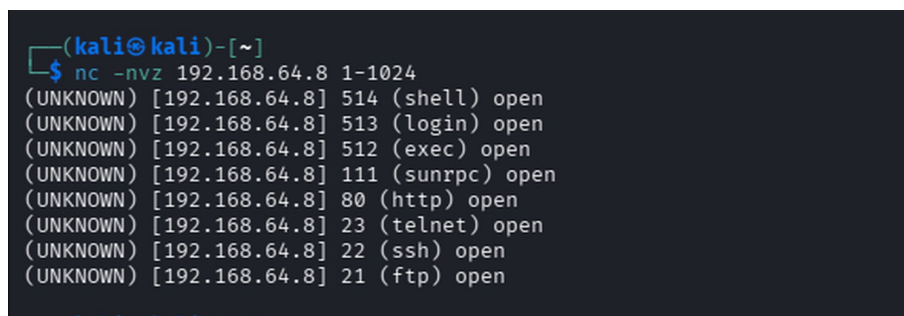


Figura 11. netcat in modalità scanner (-z): elenco delle porte aperte nel range 1-1024.

4.3 masscan, scanner asincrono ad alta velocità (comando n14)

masscan è uno scanner progettato per analizzare grandi reti a velocità elevata. Il comando qui esegue una scansione con banner grabbing su tre porte.

```
$ sudo masscan 192.168.64.8 -p21,22,80 --banners
```

masscan avvia una SYN Stealth Scan e individua le porte richieste (**found=3**). La differenza rispetto a Nmap emerge nella cattura Wireshark: masscan invia i pacchetti SYN in modo asincrono, cioè li spara

in rapidissima sequenza senza attendere le risposte una per una, usando la stessa porta sorgente.

```
(kali㉿kali)-[~]
$ sudo masscan 192.168.64.8 -p21,22,80 --banners
[sudo] password for kali:
Starting masscan 1.3.2 (http://bit.ly/14GZzcT) at 2026-09-21 14:58:57
Initiating SYN Stealth Scan
Scanning 1 hosts [3 ports/host]
Discovered open port 22/tcp on 192.168.64.8
Discovered open port 80/tcp on 192.168.64.8
Discovered open port 21/tcp on 192.168.64.8
Rate: 0.00-kpps, 100.00% done, waiting -57-secs, found=3
```

Figura 12. masscan: SYN Stealth Scan che individua le tre porte richieste (found=3).

Masscan: SYN verso 22/80/21 in rapida sequenza, stessa porta sorgente (42737)						
No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	fe80::3466:91ff:fe8...	ff02::1	ICMPv6	142	Router Advertisement from 36:66:91:8e:9a:64
2	27.310669367	b6:8d:ac:be:cf:dd	Broadcast	ARP	60	Who has 192.168.64.1? Tell 192.168.64.3
3	27.311276578	36:66:91:8e:9a:64	b6:8d:ac:be:cf:dd	ARP	42	192.168.64.1 is at 36:66:91:8e:9a:64
4	27.337543224	192.168.64.3	192.168.64.8	TCP	54	42737 → 22 [SYN] Seq=0 Win=1024 Len=0
5	27.337751141	36:66:91:8e:9a:64	Broadcast	ARP	42	Who has 192.168.64.8? Tell 192.168.64.1
6	27.340267109	192.168.64.8	192.168.64.3	TCP	58	22 → 42737 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MS
7	27.340325693	192.168.64.3	192.168.64.8	TCP	54	42737 → 22 [RST] Seq=1 Win=0 Len=0
8	27.438978460	192.168.64.3	192.168.64.8	TCP	60	42737 → 22 [ACK] Seq=1 Ack=1 Win=1200 Len=0
9	27.438989668	192.168.64.3	192.168.64.8	TCP	54	42737 → 80 [SYN] Seq=0 Win=1024 Len=0
10	27.438990793	192.168.64.3	192.168.64.8	TCP	54	42737 → 21 [SYN] Seq=0 Win=1024 Len=0

Figura 13. Wireshark — masscan: i SYN verso 22/80/21 partono in rapida sequenza dalla stessa porta sorgente, con invio asincrono (non attende le risposte in ordine).

4.4 Integrazione con Metasploit (db_import, comando n12)

L'ultimo strumento della lista non è uno scanner, ma un ponte verso le fasi successive di un penetration test: **db_import** importa i risultati di una scansione Nmap nel **database di Metasploit**, dove diventano interrogabili e riutilizzabili. Prima si esporta la scansione in formato XML, poi la si importa dentro **msfconsole**.

```
$ sudo nmap -sV -oX scan_meta.xml 192.168.64.8 # esporta in XML
$ sudo msfconsole

msf > db_status # verifica connessione DB

msf > db_import scan_meta.xml # importa lo scan

msf > hosts # elenca gli host

msf > services # elenca i servizi
```

Dopo l'inizializzazione del database (`msfdb init`), l'import ha successo e i comandi `hosts` e `services` restituiscono l'host e l'intero elenco dei servizi ora archiviati nel database di Metasploit.

```
msf > db_status
[*] Connected to msf. Connection type: postgresql.
msf > db_import scan_meta.xml
[*] Importing 'Nmap XML' data
[*] Import: Parsing with 'Nokogiri v1.14.5'
[*] Importing host 192.168.64.8
/usr/share/metasploit-framework/vendor/bundle/ruby/3.3.0/gems/recog-3.1.26/lib/recog/fingerprint/repression
[*] Successfully imported /home/kali/scan_meta.xml
msf > hosts
Hosts
=====
address      mac              name      os_name  os_flavor  os_sp  purpose  info  comments
-----
192.168.64.8  96:81:9e:81:d6:03  Linux    Linux    8ubuntu1   2.0    server

msf > services
Services
=====
host      port  proto  name      state  info  resource
-----
192.168.64.8  21    tcp    ftp        open   vsftpd 2.3.4  {}
192.168.64.8  22    tcp    ssh        open   OpenSSH 4.7p1 Debian 8ubuntu1 protocol 2.0  {}
192.168.64.8  23    tcp    telnet     open   Linux telnetd  {}
192.168.64.8  80    tcp    http       open   Apache httpd 2.2.8 (Ubuntu) DAV/2  {}
192.168.64.8  111   tcp    rpcbind    open   2 RPC #100000  {}
192.168.64.8  512   tcp    exec       open   netkit-rsh rexecd  {}
192.168.64.8  513   tcp    login      open   {}
192.168.64.8  514   tcp    tcpwrapped open   {}
192.168.64.8  1099  tcp    java-rmi   open   GNU Classpath grmiregistry  {}
192.168.64.8  1524  tcp    bindshell  open   Metasploitable root shell  {}
192.168.64.8  2121  tcp    ftp        open   ProFTPD 1.3.1  {}
192.168.64.8  3306  tcp    mysql       open   MySQL 5.0.51a-3ubuntu5  {}
192.168.64.8  5432  tcp    postgresql open   PostgreSQL DB 8.3.0 - 8.3.7  {}
192.168.64.8  5900  tcp    vnc         open   VNC protocol 3.3  {}
192.168.64.8  6000  tcp    x11         open   access denied  {}
192.168.64.8  6667  tcp    irc         open   UnrealIRCd  {}
192.168.64.8  8009  tcp    ajp13       open   {}
192.168.64.8  8180  tcp    http        open   Apache Tomcat/Coyote JSP engine 1.1  {}
```

Figura 14. Metasploit: `db_import` dello scan XML, con `hosts` e `services` che mostrano il bersaglio e i suoi servizi importati nel database.

5. Analisi del traffico: la SYN con Wireshark

Per comprendere a fondo come funziona la scansione SYN, la tecnica alla base di quasi tutti gli strumenti usati, il traffico è stato catturato con Wireshark durante una `nmap -ss`. L'analisi rende visibile il meccanismo “half-open” che dà il nome alla tecnica.

Isolando il dialogo su una singola porta aperta (la 21), si osserva la sequenza esatta: **(1)** Kali invia un pacchetto **SYN**; **(2)** la porta, essendo aperta, risponde con **SYN, ACK**; **(3)** a questo punto Kali, invece di completare la connessione con l'ACK finale (come farebbe una connessione normale), invia un **RST** che la chiude subito. La connessione TCP non viene mai stabilita del tutto: è “mezza aperta” (*half-open*), ed è proprio questo che rende la SYN scan rapida.



Figura 15. Wireshark — SYN scan (half-open) sulla porta 21: SYN → SYN,ACK (porta aperta) → RST. La connessione non viene mai completata, da cui il nome “half-open”.

Questa stessa firma (SYN → SYN,ACK → RST) è stata ritrovata nelle catture di `hping3` e negli altri strumenti: la conferma che scanner diversi implementano la medesima tecnica di base, distinguibili solo per dettagli come la gestione della porta sorgente o la velocità di invio.

6. Riepilogo delle informazioni trovate

Come richiesto dalla traccia, questa sezione finale riassume le informazioni raccolte sul bersaglio. La scansione ha delineato il profilo di Metasploitable, un sistema progettato per essere vulnerabile, che espone un numero elevato di servizi, molti obsoleti o insicuri. La tabella seguente raccoglie i servizi principali con porta, versione e note di sicurezza.

Porta	Servizio	Versione	Note di sicurezza
21/tcp	FTP	vsftpd 2.3.4	Versione con backdoor storica nota.
22/tcp	SSH	OpenSSH 4.7p1	Datato; accesso remoto.
23/tcp	Telnet	Linux telnetd	Protocollo in chiaro: credenziali non cifrate.
80/tcp	HTTP	Apache 2.2.8 (DAV/2)	Web server datato.
111/tcp	rpcbind	RPC #100000	Servizio RPC.
512-514	r-services	exec / login / shell	netkit rsh/rexec: autenticazione debole, in chiaro.
1099/tcp	Java RMI	GNU Classpath	Registro RMI esposto.
1524/tcp	bindshell	Metasploitable root shell	Shell di root già in ascolto: critico.
2121/tcp	FTP	ProFTPD 1.3.1	Secondo servizio FTP.
3306/tcp	MySQL	5.0.51a	Database esposto in rete.
3632/tcp	distccd	distcc v1 (GNU 4.2.4)	Compilazione distribuita: vulnerabilità note.
5432/tcp	PostgreSQL	8.3.0-8.3.7	Database esposto.
5900/tcp	VNC	protocol 3.3	Accesso desktop remoto.
6000/tcp	X11	(access denied)	Server grafico in ascolto.
6667/6697	IRC	UnrealIRCd	Versione con backdoor nota.
8009/tcp	AJP13	Apache Jserv	Connettore Tomcat.
8180/tcp	HTTP	Apache Tomcat/Coyote 1.1	Secondo web server (JSP).
8787/tcp	Ruby DRb	RMI (Ruby 1.8)	Oggetti remoti Ruby: vulnerabilità note.

Sintesi. Il bersaglio presenta un quadro di sicurezza gravemente compromesso, tipico di Metasploitable: protocolli in chiaro (Telnet, r-services, FTP) che espongono le credenziali; software datati con vulnerabilità pubbliche (vsftpd 2.3.4, UnrealIRCd, distccd, Java RMI); database accessibili dalla rete (MySQL, PostgreSQL); una shell di root già in ascolto sulla porta 1524. In un contesto reale, ciascuno di questi punti costituirebbe un vettore di attacco diretto, questo rende Metasploitable la palestra ideale per esercitarsi.